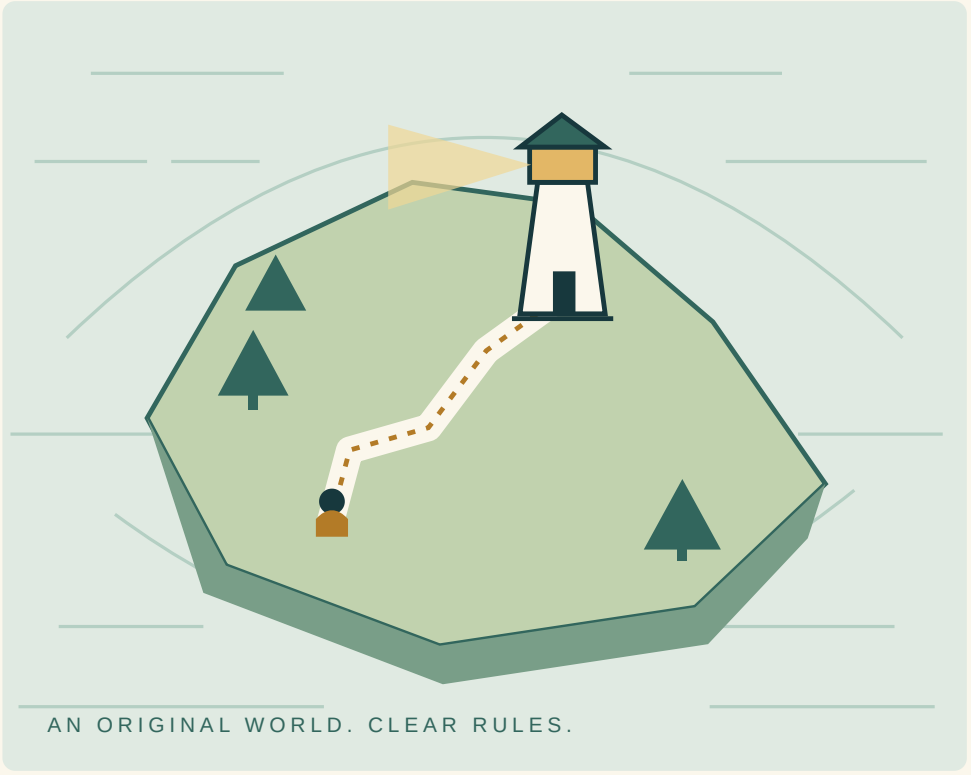
01 / THE IDEA

The Great Plot Twist

A text mystery becomes a shared world.

> **The problem**
>
> Make a text adventure easy to enter, explore and play together.

Our game: an island mystery in the browser. Original pixel art. Clear choices. Python rules.



AN ORIGINAL WORLD. CLEAR RULES.

Explore. Investigate. Decide.

A place invites a question. A choice carries the story forward.

01

Travel

Choose a destination on the sea chart.

02

Investigate

Walk, notice and interact with the world.

03

Choose

Use clues and dialogue to make a decision.

04

Continue

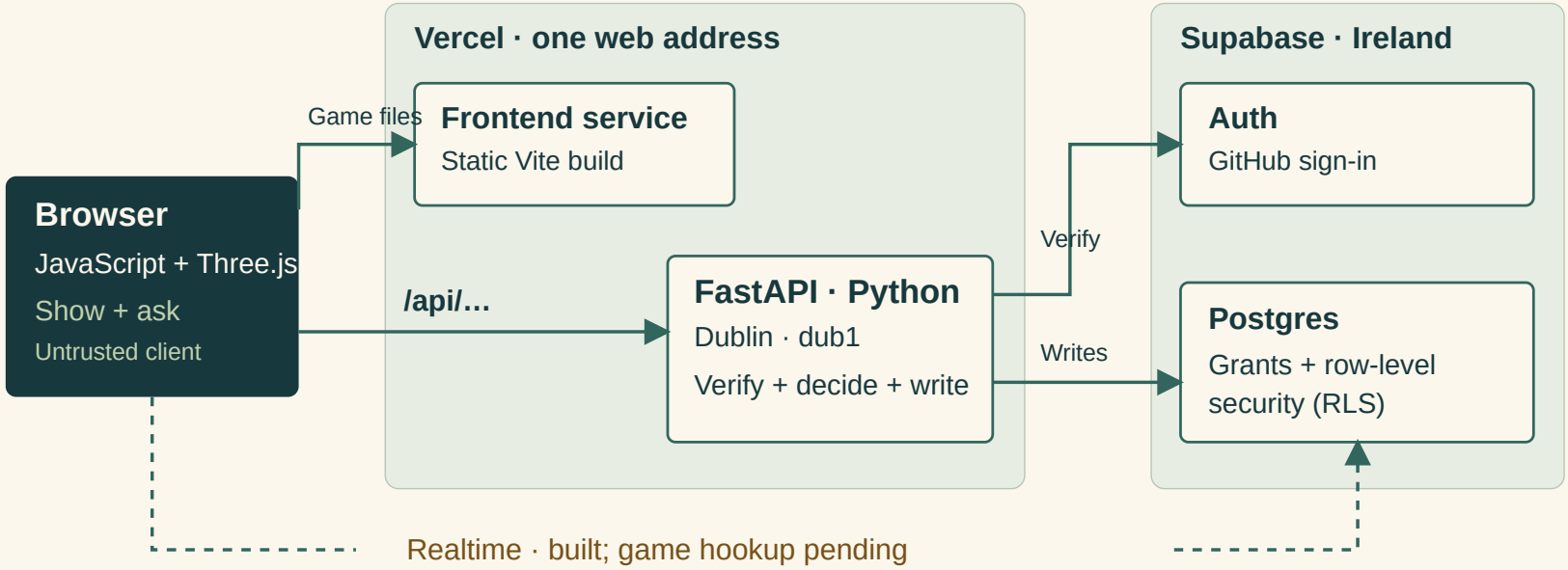
Carry progress into the next visit.

Now Walking, chart travel, original intro and local save.

Target Connected crews, a shared case and a play-money economy. Screen integration remains.

Source: frontend/GRAPHICS.md · current game and adapter implementation

Three parts, one trust boundary.



Browser: show and ask. Server: verify and decide. Database: enforce constraints.

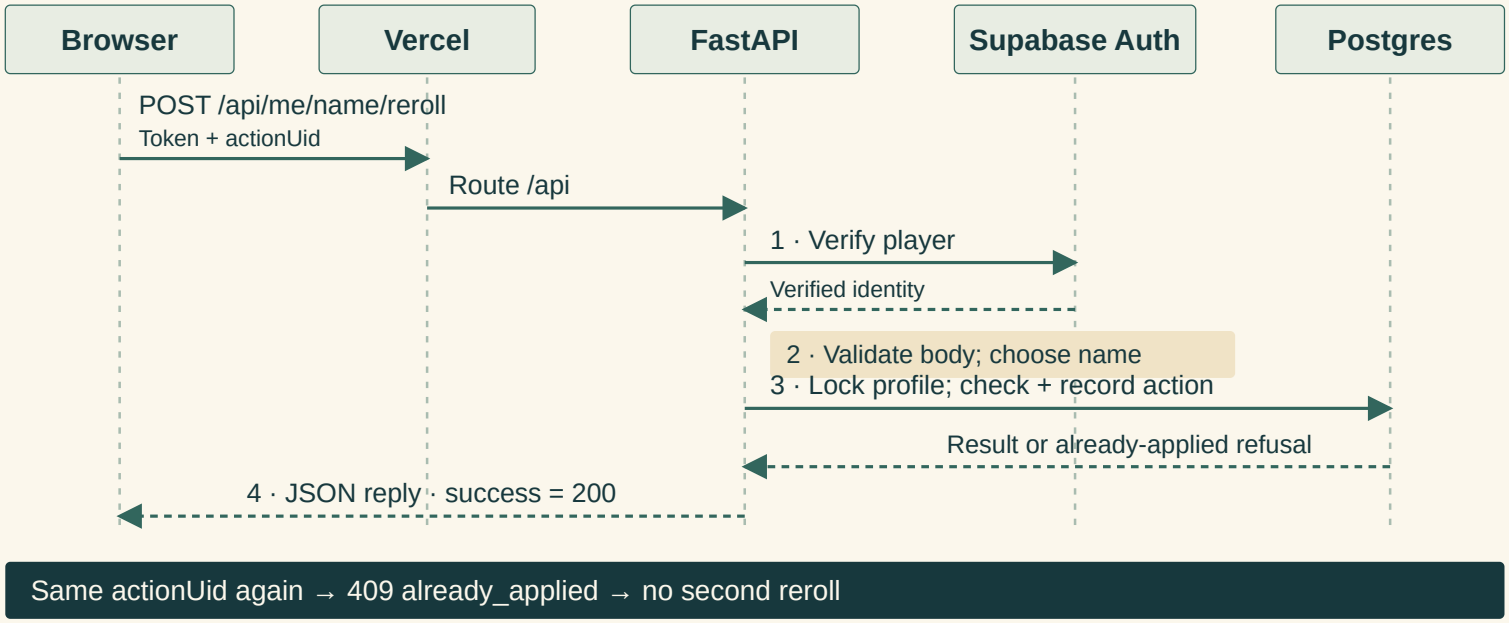
The browser can also sign in and read rows directly where grants and RLS allow. Game writes belong behind the API.

Sources: docs/ARCHITECTURE.md · docs/INTERFACES.md · vercel.json · main 60f7fd0

04 / ONE WRITE, END TO END

Follow one request.

Implemented API example: change a generated captain name. Account UI is built; real transport integration remains.



Sources: backend/server/routes/captain_name.py · server/auth.py · accounts SQL migration

The server decides.
The database guards.

Verified identity → Valid intent → Python rules → Atomic write

Grants

May this role use the table at all?

Row-level security

Which rows may this role access?

Stable action references

The same action cannot apply its effect twice.

The privileged server key bypasses RLS. Server authorization still has to be correct.

Sources: D003 / D004 · supabase/README.md · wallet and account rules/tests

Protect the story and the browser.

01 · CONTENT

Keep the solution server-side

The build guard checks current reveal fingerprints and elimination data in built files.

02 · BROWSER

Limit exposure

Account replies: private, no-store. Sign-in check page: a Content Security Policy.

03 · HARDENING

Security headers configured

Sitewide CSP, framing protection and browser policies. Verify deployed behavior next.

A targeted guard is useful evidence, not proof that every spoiler or browser risk is covered.

Sources: check-spoilers.js · public/auth-check.html · account routes · vercel.json / server/app.py

07 / THE EVIDENCE

7,294 tests, with a clear limit.

6,408

Python tests · pytest

886

JavaScript tests · 72 Vitest files

Ruff passed | Production build passed | 99 files / 12 protected lines checked

Repeated wallet actions. Stale saves. Python ↔ JavaScript story parity.

Counts include parameterized cases, rather than complete user journeys. Controlled service fakes do not establish live Supabase RLS or production readiness.

Rerun 6 Nov 2026 · origin/main 60f7fd0 · commands, exact evidence and limits in sources.md

08 / HOW WE WORKED WITH AI

AI accelerated the work. Gates controlled integration.

01

Human scope

02

Bounded agent task

03

Tests, build, scan

04

Independent agent review

05

Claude integration

D015, owner-authorized: Claude integrates Claude/Codex packages after tests, style, build, secret scan and independent agent review. Database changes add Opus review and a live-database check.

Nils reviews at the end or ad hoc. Teammates' own contributions retain human peer review.

Sources: D015, extended 6 Nov 2026 · CONTRIBUTING.md · AGENTS.md

What we learned to explain.

01

Put rules behind a boundary

A wallet screen can change. The authority to move coins cannot move into the browser.

02

Design for the retry

A timeout is not an outcome. Stable action references make recovery explicit.

03

Keep agent work reviewable

Small scopes, shared contracts and independent checks expose integration gaps.

A convincing screen is a beginning.
A defined, tested behavior is progress.

Examples: wallet references · cached-source fallbacks · bounded task and review rules

Connect. Verify. Rehearse.

CONNECT

Finish the real flows

Real account/crew transport, case integration and the built Realtime hookup.

VERIFY

Test the live boundaries

RLS, concurrency, production behavior and configured security headers.

REHEARSE

Pair with the G11 demo

Show the experience, with a plan for service failures.

The demo shows the island.
These decisions make it a program we can explain.

Next steps, not completed claims - no case solution shown